

Explain Neural Network From Scratch

Nguyen Thanh Dat

October 2026

1 Introduction

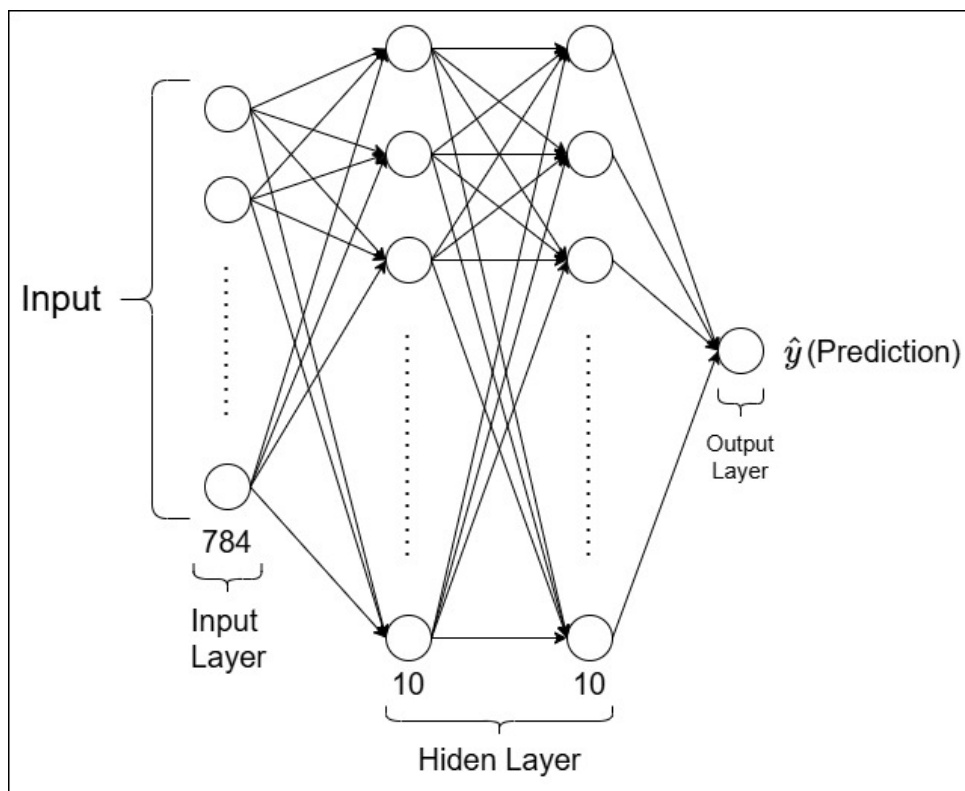
This is a document I compiled for revision and to learn AI concepts starting from the foundational mathematical principles. As this is my first piece of writing, there may inevitably be mistakes. I kindly ask for your understanding, and any feedback or suggestions for improvement are warmly welcomed at datnt23.work@gmail.com.

In this document, I will implement a neural network from scratch without relying on frameworks like PyTorch or TensorFlow, using only NumPy, Matplotlib, and Pandas.

Note: To understand this material, readers are expected to be familiar with basic mathematical concepts such as derivatives, partial derivatives, the chain rule, and linear algebra.

2 Neural Network Architecture

For this study, the MNIST dataset is employed as the benchmark. The corresponding neural network architecture is outlined below:



Hình 1: Neural Network Architecture

Since the MNIST dataset consists of 28x28 grayscale images, each input image is flattened into 784 features. As shown in the architecture above, the input layer comprises 784 neurons.

In this document, I employ a simple neural network architecture consisting of four layers: an input layer with 784 neurons, two hidden layers with 10 neurons each, and an output layer with a single neuron that outputs the predicted digit.

3 Forward Propagation, Backpropagation, and Their Mathematical Explanations

In this section, I will detail how the input data is propagated through the network and how the computations are performed.

First, consider a single image, represented as: $x \in \mathbb{R}^{784 \times 1}$

– **Layer 1:** $z_1 = W_1 x + b_1$

with $W_1 : (10, 784), b_1 : (10, 1)$ so $z_1 : (10, 1)$

after ReLU: $a_1 = \text{ReLU}(z_1)$

– **Layer 2:** $z_2 = W_2 a_1 + b_2$

with $W_2 : (10, 10), b_2 : (10, 1)$ so $z_2 : (10, 1)$

and: $a_2 = \text{softmax}(z_2)$

Therefore, a_2 represents the probability distribution over the 10 classes:

$$a_2 = \begin{bmatrix} P(y = 0 | x) \\ P(y = 1 | x) \\ \vdots \\ P(y = 9 | x) \end{bmatrix}$$

Assuming the true image is the digit 3, the ground-truth target vector $y \in \mathbb{R}^{10 \times 1}$ is one-hot encoded as:

$$y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \leftarrow \text{class 0} \\ \leftarrow \text{class 1} \\ \leftarrow \text{class 2} \\ \leftarrow \text{class 3} \\ \leftarrow \text{class 4} \\ \vdots \\ \leftarrow \text{class 9} \end{matrix}$$

We have, cross-entropy:

$$L = - \sum_{k=0}^9 y_k \log(a_{2,k}) \quad (1)$$

Since there is only one $y_k = 1$, if the class is indeed c :

$$L = -\log(a_{2,c}) \quad (2)$$

The purpose of the backward propagation is to determine: $\frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}$ to then update the weights using gradient descent:

$$\begin{aligned} W &\leftarrow W - \alpha \frac{\partial L}{\partial W} \\ b &\leftarrow b - \alpha \frac{\partial L}{\partial b} \end{aligned} \quad (3)$$

where, α : learning rate.

Backward propagation:

We have: $a_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$ and $L = -\sum_i y_i \log(a_i)$

$$\Rightarrow L = -\sum_i y_i \log\left(\frac{e^{z_i}}{\sum_j e^{z_j}}\right) \quad (4)$$

$$\Rightarrow L = -\sum_i y_i \left[\log(e^{z_i}) - \log\left(\sum_j e^{z_j}\right) \right] = -\sum_i y_i \left[z_i - \log\left(\sum_j e^{z_j}\right) \right] \quad (5)$$

Since one-hot: $\sum_i y_i = 1$ so,

$$L = -\sum_i y_i z_i + \log\left(\sum_j e^{z_j}\right) \quad (6)$$

Derivative the equation (6) with respect to z_k

$$\begin{aligned} \frac{\partial L}{\partial z_k} &= \frac{\partial}{\partial z_k} \left(-\sum_i y_i z_i \right) + \frac{\partial}{\partial z_k} \log\left(\sum_j e^{z_j}\right) \\ &= -y_k + \frac{1}{\sum_j e^{z_j}} e^{z_k} \end{aligned} \quad (7)$$

$$= -y_k + a_k = a_k - y_k \text{ (because : } \frac{1}{\sum_j e^{z_j}} e^{z_k} = a_k \text{)}$$

$$\Rightarrow \frac{\partial L}{\partial z_k} = a_k - y_k \quad (8)$$

For all 10 output:

$$\boxed{\frac{\partial L}{\partial z_2} = a_2 - y} \quad (9)$$

We have: $z_2 = W_2 a_1 + b_2$

Considering each neuron individually:

$$z_{2,i} = \sum_j W_{2,ij} a_{1,j} + b_{2,i} \quad (10)$$

According to Chain rule:

$$\frac{\partial L}{\partial W_{2,ij}} = \frac{\partial L}{\partial z_{2,i}} \frac{\partial z_{2,i}}{\partial W_{2,ij}} \quad (11)$$

Since $\frac{\partial z_{2,i}}{\partial W_{2,ij}} = a_{1,j}$:

$$\begin{aligned} \frac{\partial L}{\partial W_{2,ij}} &= \frac{\partial L}{\partial z_{2,i}} a_{1,j} \\ &= (a_{2,i} - y) a_{1,j} \end{aligned} \quad (12)$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial W_2} = (a_2 - y) a_1^T} \quad (13)$$

And,

$$\begin{aligned}\frac{\partial L}{\partial b_{2,i}} &= \frac{\partial L}{\partial z_{2,i}} \frac{\partial z_{2,i}}{\partial b_{2,i}} \\ &= (a_{2,1} - y) \left(\text{because } \frac{\partial z_{2,i}}{\partial b_{2,i}} = 1 \right)\end{aligned}\tag{14}$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial b_2} = a_2 - y}\tag{15}$$

Now, the gradient should backward to layer 1.

First, we determine : $\frac{\partial L}{\partial z_1}$. According to Chain rule:

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \frac{\partial a_1}{\partial z_1}\tag{16}$$

We have: $z_2 = W_2 a_1 + b_2$ Considering: $z_{2,i} = \sum_j W_{2,ij} a_{1,j} + b_{2,i}$

We want to determine gradient with $a_{1,j}$, and $a_{1,j}$ affect to all neuron output, so:

$$\frac{\partial L}{\partial a_{1,j}} = \sum_i \frac{\partial L}{\partial z_{2,i}} \frac{\partial z_{2,i}}{\partial a_{1,j}}\tag{17}$$

but: $\frac{\partial z_{2,i}}{\partial a_{1,j}} = W_{2,ij}$ so,

$$\frac{\partial L}{\partial a_{1,j}} = \sum_i W_{2,ij} (a_{2,i} - y_i)\tag{18}$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial a_1} = W_2^T (a_2 - y)}\tag{19}$$

We have:

$$a_1 = \text{ReLU}(z_1) = \begin{cases} z_1, & \text{when } z_1 > 0 \\ z_1, & \text{khi } z_1 \leq 0 \end{cases}\tag{20}$$

Derivative of ReLU:

$$\text{ReLU}'(z) = \begin{cases} 1, & \text{khi } z > 0 \\ 0, & \text{khi } z \leq 0 \end{cases}\tag{21}$$

We want to calculate $\frac{\partial L}{\partial z_1}$, using Chain rule:

$$\begin{aligned}\frac{\partial L}{\partial z_1} &= \frac{\partial L}{\partial a_1} \frac{\partial a_1}{\partial z_1} \\ &= \frac{\partial L}{\partial a_1} \odot \text{ReLU}'(z_1)\end{aligned}\tag{22}$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial z_1} = W_2^T (a_2 - y) \odot \text{ReLU}'(z_1)}\tag{23}$$

where $\odot$ is element – wise multiplication.

We have: $z_1 = W_1x + b_1$, using Chain rule:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial W_1} \quad (24)$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial W_1} = W_2^T (a_2 - y) \odot ReLU'(z_1) x^T} \quad (25)$$

Similarly,

$$\begin{aligned} \frac{\partial L}{\partial b_1} &= \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b_1} \\ &= \frac{\partial L}{\partial z_1} (\text{because } \frac{\partial z_1}{\partial b_1} = 1) \end{aligned} \quad (26)$$

$$\Rightarrow \boxed{\frac{\partial L}{\partial b_1} = W_2^T (a_2 - y) \odot ReLU'(z_1)} \quad (27)$$

In practice, during computation, we do not evaluate a single image alone, but rather the entire training set. We change from single image to 'm' image. The actual loss is usually averaged:

$$J = \frac{1}{m} \sum_{s=1}^m L^{(s)} \quad (28)$$

Therefore,

$$\frac{\partial J}{\partial W_2} = \frac{1}{m} \sum_{s=1}^m \frac{\partial L^{(s)}}{\partial W_2} \quad (29)$$

with single sample, we have:

$$\frac{\partial L^{(s)}}{\partial W_2} = (a_2 - y)^{(s)} (a_1^{(s)})^T \quad (30)$$

We group the m samples into columns: $(a_2 - y) : (10, m)$, $a_1 : (10, m)$

This matrix multiplication automatically sums the contributions of all m samples, so:

$$\boxed{\frac{\partial J}{\partial W_2} = \frac{1}{m} (a_2 - y) a_1^T} \quad (31)$$

Similary, we have: $\frac{\partial J}{\partial b_2} = \frac{1}{m} \sum_{s=1}^m \frac{\partial L^{(s)}}{\partial z_2}$, and:

$$\frac{\partial J}{\partial z_2} = \begin{bmatrix} \left| \frac{\partial L^{(1)}}{\partial z_2} \right| & \left| \frac{\partial L^{(2)}}{\partial z_2} \right| & \dots & \left| \frac{\partial L^{(3)}}{\partial z_2} \right| \\ \left| \right| & \left| \right| & & \left| \right| \end{bmatrix} \text{ with shape: } (10, m) \quad (32)$$

We need to sum across the samples, that is sum horizontally row by row:

$$\boxed{\frac{\partial J}{\partial b_2} = \frac{1}{m} \sum_{columns} \frac{\partial J}{\partial z_2} = \frac{1}{m} \sum_{columns} (a_2 - y)} \quad (33)$$

Similary, we have:

$$\frac{\partial J}{\partial W_1} = \frac{1}{m} W_2^T (a_2 - y) \odot ReLU'(z_1) x^T \quad (34)$$

$$\frac{\partial J}{\partial b_1} = \frac{1}{m} \sum_{columns} (W_2^T (a_2 - y) \odot ReLU'(z_1)) \quad (35)$$

4 Conclude

Below is a summary of the neural network:

Forward:

$$X \xrightarrow{W_1, b_1} Z_1 \xrightarrow{ReLU} A_1 \xrightarrow{W_2, b_2} Z_2 \xrightarrow{Softmax} A_2 \xrightarrow{CE} L \quad (36)$$

Backward:

$$L \rightarrow \frac{\partial J}{\partial z_2} \rightarrow \frac{\partial J}{\partial W_2}, \frac{\partial J}{\partial b_2} \rightarrow \frac{\partial J}{\partial a_2} \rightarrow \frac{\partial J}{\partial z_1} \rightarrow \frac{\partial J}{\partial W_1}, \frac{\partial J}{\partial b_1} \quad (37)$$

After computing the gradients, we update the weights using the following formula:

$$\begin{aligned} W_1 &\leftarrow W_1 - \alpha \frac{\partial J}{\partial W_1} \\ b_1 &\leftarrow b_1 - \alpha \frac{\partial J}{\partial b_1} \\ W_2 &\leftarrow W_2 - \alpha \frac{\partial J}{\partial W_2} \\ b_2 &\leftarrow b_2 - \alpha \frac{\partial J}{\partial b_2} \end{aligned} \quad (38)$$

Thank you for taking the time to read through this document. The above sections present my mathematical breakdown of neural networks. While I have strived for accuracy, some imperfections may remain. Any constructive feedback or suggestions sent to my personal email would be warmly welcomed. The complete source code and dataset for this neural network are available at the following links:

- **Source Code:** [GitHub Repository](#)
- **Dataset:** [Google Drive Folder](#)